TCMM — Testing & Coverage Management Module

TCMM — Testing & Coverage Management Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Quality Standard (SQS)

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Core Module

Governed Space: Software Testing & Coverage Governance

Module ID: SLGA-SQS-TCMM-003

Origin ID: OOF.CM.SLGA.05.03.01

Version: 1.0

Status: Canonical · Open Standard

Effective Date: 12.10.2026

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Testing & Coverage Management Module (TCMM) governs the

identification, planning, design, authorization, execution, observation, measurement, verification, evidence integrity, coverage assessment, and lifecycle continuity of software quality testing.

TCMM establishes the methodological conditions under which software testing

activities and coverage claims can be recognized as identifiable, appropriately scoped, competently executed, evidence-supported, and traceable to the software artifact, configuration, quality criteria, and operating conditions actually assessed.

It governs the relationship between governed quality criteria, test

objectives, test cases, test inputs, expected behavior, test environments, execution authority, actual execution records, observed software behavior, coverage measurements, test results, testing limitations, and the evidence supporting testing conclusions.

TCMM distinguishes test design from test execution, test execution from

verified results, test success from demonstrated software quality, numerical coverage from meaningful behavioral coverage, and testing evidence from broader quality acceptance.

It applies to testing performed by humans, organizations, automated testing

infrastructure, AI coding assistants, autonomous testing agents, and software quality assessment systems.

TCMM is technology-independent and does not prescribe a particular testing framework, programming language, test automation platform, or software development methodology.

Its principal output is:

Governed Software Testing & Coverage State

Its foundational distinction is:

Test Execution Success ≠ Demonstrated Software Quality.

Operational Role

The operational role of TCMM is:

To ensure that software testing and coverage claims remain identifiable, bounded by governed quality criteria, connected to the software and environment actually evaluated, supported by competent execution evidence, and interpreted according to their demonstrated scope and limitations without confusing successful test execution with comprehensive software quality verification.

TCMM establishes the testing and coverage governance mechanism of SQS.

It determines whether material testing activities were appropriately

defined, authorized, executed, observed, recorded, and interpreted.

It also determines what those activities actually cover and which relevant

quality conditions remain untested, insufficiently tested, or unresolved.

TCMM does not independently establish software quality requirements, redefine quality criteria, resolve defects, authorize quality acceptance, certify software security, or authorize release and deployment.

Operational Question

Which software behaviors, quality conditions, artifacts, configurations,

and operating scenarios have actually been tested, under which methods and execution conditions, with what evidence and coverage limitations, and what can legitimately be concluded from the observed results?

Position in Architecture Governance Flow

The SLGA™ governance flow is:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting:

Software Lifecycle Control

TCMM operates within:

Software Quality Standard (SQS)

The SQS internal governance flow is:

Quality Requirements → Quality Criteria Definition → Testing & Coverage

Management → Defect Management → Quality Conformity & Acceptance

TCMM governs:

Testing & Coverage Management

It is the third Core Module of SQS.

Its principal upstream references are:

Governed Software Quality Requirements State

from:

QRM — Quality Requirements Module

Governed Quality Criteria Definition State

from:

QCDM — Quality Criteria Definition Module

Relevant supporting references include:

Governed Software Requirements Baseline

from Software Requirements Standard (SRS).

Governed Software Creation State

from Software Creation Standard (SCS).

Governed Software Dependency State

from Software Dependency Standard (SDS).

Governed Software Configuration State

from Software Configuration Standard (SCFS).

Its principal output is:

Governed Software Testing & Coverage State

This output supports:

DFM — Defect Management Module

QCAM — Quality Conformity & Acceptance Module

and contributes to:

Governed Software Quality State

TCMM may be invoked repeatedly as software artifacts, configurations, dependencies, quality criteria, test environments, or relevant operational conditions change.

• Governed Space

Software Testing & Coverage Governance

This space governs the conditions under which testing activities and

coverage claims become sufficiently established for competent software quality assessment.

Its principal dimensions include:

• Testing governance object identity

• Testing purpose

• Testing scope

• Software object correspondence

• Artifact identity

• Artifact version correspondence

• Configuration correspondence

• Quality requirement correspondence

• Quality criterion correspondence

• Test strategy

• Test classification

• Test objective

• Test design

• Test case identity

• Test scenario definition

• Test input governance

• Test data governance

- Expected behavior
- Test oracle
- Test preconditions
- Test environment
- Environment suitability
- Environment correspondence
- Test dependencies
- Test execution authority
- Automated testing authority
- Test execution identity
- Test execution conditions
- Test execution evidence
- Test completion
- Test interruption
- Test failure
- Test invalidity
- Observed software behavior
- Expected–observed correspondence
- Test result classification
- Test result verification
- Test repeatability
- Test determinism
- Test flakiness
- Test coverage basis
- Requirements coverage
- Functional coverage
- Code coverage
- Branch coverage
- Integration coverage
- Scenario coverage
- Environment coverage
- Failure-condition coverage
- Coverage measurement
- Coverage interpretation
- Coverage exclusions
- Coverage gaps

• Critical coverage gaps

• Test evidence competence

• Test evidence integrity

• Test uncertainty

• Test conflicts

• Regression testing

• Testing continuity

• Testing reassessment

• Testing governance states

This governed space remains distinct from quality criteria definition under

QCDM, defect resolution under DFM, and quality conformity and acceptance under QCAM.

Testing Governance Object

Every material TCMM assessment shall establish a Testing Governance Object.

The object represents an identifiable software testing activity or testing

assessment and its applicable governance conditions.

It should identify:

Testing Identity: The identifiable testing activity.

Testing Purpose: The reason testing is performed.

Software Object: The software being tested.

Artifact Identity: The relevant software artifact or version.

Configuration Reference: The configuration being evaluated.

Quality Criteria: The criteria against which testing is performed.

Testing Scope: The applicable assessment boundaries.

Test Method: The method used.

Test Cases: The defined tests.

Test Environment: The relevant execution conditions.

Execution Authority: The legitimate basis for testing actions.

Execution Evidence: The records of actual test execution.

Observed Results: The behavior or measurements obtained.

Coverage State: The recognized scope of testing.

Testing Limitations: Known exclusions and unresolved conditions.

Governance State: The recognized testing condition.

The testing governance object shall remain distinguishable from a test

suite, test report, test run, or coverage dashboard.

Testing Governance Object ≠ Test Execution Report.

Testing Purpose Governance

TCMM shall identify the purpose of material testing activities.

Purposes may include:

• Functional verification

• Requirements conformity

• Integration assessment

• Reliability evaluation

• Performance assessment

• Compatibility testing

• Regression testing

• Recovery assessment

• Quality evidence generation

• Defect reproduction

• Defect resolution verification

Testing purpose shall remain connected to governed quality criteria.

Testing Purpose ≠ Testing Result.

Testing Scope Governance

TCMM shall establish the boundaries of material testing activities.

Scope may concern:

• Software functions

• Components

• Interfaces

• Integrated systems

• Artifact versions

• Configurations

• Operating environments

• User scenarios

• Workloads

• Failure conditions

Scope shall distinguish tested, excluded, and unresolved conditions.

Defined Testing Scope ≠ Executed Testing Scope.

Software Object Correspondence

TCMM shall identify the software object to which testing evidence applies.

Objects may include:

• Individual function

• Software component

• Library

• Application

• Service

• Interface

• Embedded software

• Integrated system

Testing evidence shall not be generalized to materially different software

objects without competent justification.

Testing One Software Component ≠ Testing the Entire Software System.

Artifact Identity Governance

TCMM shall identify the software artifact actually tested.

Relevant information may include:

• Artifact identity

• Content reference

• Build reference

• Artifact version

• Relevant provenance

• Integrity reference

A declared version label shall not independently establish the identity of

the tested artifact.

Declared Tested Version ≠ Verified Tested Artifact.

Artifact Version Correspondence

TCMM shall preserve the relationship between testing evidence and the

software version actually evaluated.

Testing performed against one version shall not automatically establish the

quality of a different version.

Material differences shall remain explicit.

Testing Version A ≠ Verification of Version B.

Configuration Correspondence

TCMM shall preserve the relationship between test results and the software

configuration under which they were obtained.

Relevant conditions may include:

• Configuration baseline

• Runtime settings

• Component versions

• Build parameters

• Feature flags

• Relevant external services

Test results shall remain bounded by their applicable configuration.

Tested Configuration ≠ Every Configuration.

Quality Requirement Correspondence

TCMM shall preserve the relationship between testing activities and the

governed quality requirements they support.

Testing shall identify which requirements are:

• Addressed

• Partially addressed

• Not addressed

• Not applicable

• Unresolved

Requirement correspondence shall not independently establish requirement

satisfaction.

Requirement Linked to Test ≠ Requirement Verified by Test.

Quality Criterion Correspondence

TCMM shall associate material testing activities with applicable governed

quality criteria.

The relationship should preserve:

• Criterion identity

• Applicable software object

• Expected quality condition

• Assessment method

• Evaluation rule

• Relevant threshold

• Required evidence

TCMM shall not silently redefine the criterion during execution.

Executing a Test Against Modified Criteria ≠ Satisfying the Original

Criteria.

Test Strategy Governance

TCMM shall identify the strategy governing material testing activities.

A strategy may establish:

• Testing objectives

• Assessment levels

• Relevant methods

• Coverage expectations

• Test sequencing

• Required environments

• Evidence requirements

• Relevant limitations

A documented strategy shall remain distinguishable from its execution.

Test Strategy Approval ≠ Test Strategy Execution.

Test Classification

TCMM shall classify material testing activities according to their purpose

and scope.

Classifications may include:

• Unit testing

• Component testing

• Integration testing

• System testing

• Functional testing

• Performance testing

• Reliability testing

• Compatibility testing

• Regression testing

• Recovery testing

• Acceptance-support testing

Classification shall not independently establish test suitability.

Test Classification ≠ Test Competence.

Test Objective Governance

TCMM shall identify the objective of each material test or test group.

An objective should establish:

• The behavior being assessed

• The applicable criterion

• The relevant software object

• The expected result

• The intended evidentiary contribution

Objectives shall remain distinguishable from observed outcomes.

Test Objective ≠ Test Outcome.

Test Design Governance

TCMM shall preserve sufficiently identifiable test designs.

A test design may establish:

• Test case identity

• Preconditions

• Inputs

• Execution steps

• Expected results

• Environment requirements

• Evaluation method

• Evidence requirements

Test design shall remain connected to the applicable quality criterion.

Designed Test ≠ Executed Test.

Test Case Identity

TCMM shall establish identifiable references for material test cases.

A test case should remain distinguishable by:

• Identity

• Purpose

• Applicable criterion

• Software object

• Inputs

• Expected result

• Relevant version

• Revision history

Test identity shall not be confused with execution identity.

Test Case Identity ≠ Test Execution Identity.

Test Scenario Definition

TCMM shall identify relevant scenarios under which software behavior is evaluated.

Scenarios may concern:

• Normal operation

• Boundary conditions

• Invalid input

• Failure conditions

• Recovery behavior

• Concurrent operation

• Integration behavior

• Relevant user workflows

A defined scenario shall not be treated as an executed scenario.

Defined Test Scenario ≠ Observed Scenario Behavior.

Test Input Governance

TCMM shall identify material inputs used during testing.

Inputs may include:

• Valid values

• Invalid values

• Boundary values

• Input sequences

• Large datasets

• Unexpected combinations

• Relevant operational data

Test inputs shall remain associated with execution evidence.

Defined Test Input ≠ Verified Executed Input.

Test Data Governance

TCMM shall preserve relevant conditions concerning test data.

Conditions may include:

• Data identity

• Data origin

• Data structure

• Data suitability

• Data representativeness

• Data integrity

• Data limitations

Data protection and access permissions remain subject to competent external

governance.

Available Test Data ≠ Representative Test Data.

Expected Behavior Governance

TCMM shall preserve the expected software behavior relevant to each

material test.

Expected behavior shall derive from applicable governed quality criteria.

It may concern:

• Functional outputs

• Error responses

• Timing

• State changes

• Resource usage

• Recovery behavior

Expected behavior shall remain distinguishable from actual observations.

Expected Test Result ≠ Observed Test Result.

Test Oracle Governance

TCMM shall identify the mechanism used to determine whether observed

behavior satisfies the expected condition.

A test oracle may involve:

• Explicit expected output

• Reference implementation

• Formal specification

• Invariant

• Property-based condition

• Competent human evaluation

• Approved measurement threshold

The oracle shall remain sufficiently independent and suitable for the

relevant claim.

An AI-generated expected result shall not be presumed correct merely

because the same AI system generated the implementation.

Test Oracle Availability ≠ Test Oracle Validity.

Test Preconditions

TCMM shall identify material conditions required before testing can meaningfully proceed.

Preconditions may include:

• Required software state

• Environment readiness

• Test data availability

• Dependency availability

• Relevant permissions

• Required configuration

• Measurement readiness

Preconditions shall remain distinguishable from verified readiness.

Declared Test Preconditions ≠ Verified Test Readiness.

Test Environment Governance

TCMM shall identify the environment in which material tests are executed.

Relevant dimensions may include:

• Hardware

• Operating system

• Runtime

• Dependencies

• Network

• Configuration

• External services

• Data conditions

• Resource limits

Environment identity shall remain connected to test execution evidence.

Documented Test Environment ≠ Actual Test Environment.

Environment Suitability

TCMM shall assess whether the environment is suitable for the relevant test purpose.

Suitability may depend on:

• Applicable criterion

• Intended use

• Workload

• Platform characteristics

• Measurement accuracy

• Dependency behavior

• Required isolation

An environment may be technically functional while unsuitable for a particular quality claim.

Operational Test Environment ≠ Suitable Assessment Environment.

Environment Correspondence

TCMM shall preserve material differences between the test environment and

the environment relevant to the quality claim.

Differences may concern:

• Hardware capacity

• Network conditions

• Data volume

• External integrations

• Runtime configuration

• Dependency versions

• Workload distribution

Where differences materially limit interpretation, the limitation shall remain explicit.

Test Environment Similarity ≠ Demonstrated Operational Equivalence.

Test Dependency Governance

TCMM shall identify material dependencies required for testing.

Dependencies may include:

• Test frameworks

• External services

• Data sources

• Measurement tools

• Mock services

• Simulated components

• Supporting infrastructure

Dependency suitability and origin remain subject to SDS where applicable.

Available Test Dependency ≠ Suitable Test Dependency.

Test Execution Authority

TCMM shall identify the legitimate basis for material test execution

activities.

Authority may concern:

• Access to software

• Access to environments

• Execution permissions

• Use of test data

• Invocation of external services

• Modification of test infrastructure

• Automated testing actions

Authority to execute tests shall not be confused with authority to redefine

quality criteria or approve quality acceptance.

Test Execution Authority ≠ Quality Acceptance Authority.

Automated Testing Authority

TCMM shall preserve the boundaries under which automated systems and AI

agents execute testing activities.

Relevant actions may include:

• Generating tests

• Selecting test cases

• Executing test suites

• Accessing environments

• Modifying test data

• Invoking external services

• Collecting evidence

• Proposing conclusions

Authorization for a testing objective shall not automatically authorize

every constituent tool action.

Testing Task Authority ≠ Constituent Testing Action Authority.

Test Execution Identity

TCMM shall establish identifiable references for material test executions.

Execution identity should preserve:

• Test case reference

• Execution instance

• Software artifact

• Configuration

• Environment

• Execution actor

• Execution time

• Relevant inputs

• Execution result

Repeated executions of the same test case shall remain distinguishable.

Same Test Case ≠ Same Test Execution.

Test Execution Conditions

TCMM shall preserve the material conditions under which tests were actually

executed.

Conditions may include:

• Software state

• Runtime parameters

• Test inputs

• Test data

• Environment conditions

• Resource availability

• External dependencies

• Execution sequence

Declared execution conditions shall remain distinguishable from observed

execution conditions.

Planned Test Conditions ≠ Actual Test Conditions.

Test Execution Evidence

TCMM shall preserve competent evidence of material test execution.

Evidence may include:

• Execution logs

• Test result records

• Observed outputs

• Measurement records

• Error traces

• Relevant environment records

• Artifact references

• Execution timestamps

Evidence shall remain associated with the actual execution instance.

Test Report Generation ≠ Verified Test Execution.

Test Completion Governance

TCMM shall determine whether a material test execution reached its defined

completion condition.

Completion may involve:

• Execution of required steps

• Collection of required observations

• Completion of measurements

• Availability of evaluation evidence

Completion shall remain distinguishable from success.

Completed Test ≠ Passed Test.

Test Interruption Governance

TCMM shall recognize test executions interrupted before meaningful

completion.

Interruptions may arise from:

• Environment failure

• Tool failure

• Dependency unavailability

• Timeout

• Resource exhaustion

• Execution cancellation

• Permission failure

An interrupted test shall not automatically be classified as a software

quality failure.

Interrupted Test ≠ Failed Software Quality Criterion.

Test Failure Governance

TCMM shall identify test results that fail applicable evaluation conditions.

Failure may indicate:

• Observed software deviation

• Incorrect expected result

• Unsuitable test method

• Environment problem

• Invalid test data

• Test implementation defect

A failed test shall trigger competent interpretation rather than automatic

attribution to software defects.

Failed Test ≠ Verified Software Defect.

Test Invalidity Governance

TCMM shall recognize testing evidence that cannot competently support its intended conclusion.

Invalidity may arise where:

• Wrong artifact was tested

• Required preconditions were absent

• Test data was corrupted

• Environment conditions were unsuitable

• Evaluation criteria were modified without authority

• Required execution evidence is missing

An invalid test shall remain distinguishable from a valid failed test.

Invalid Test Result ≠ Valid Test Failure.

Observed Software Behavior

TCMM shall preserve the behavior actually observed during material test

executions.

Observations may include:

• Functional outputs

• Error behavior

• State transitions

• Timing

• Resource usage

• Integration responses

• Recovery behavior

• Relevant failure conditions

Observed behavior shall remain connected to the tested artifact and

execution context.

Observed Behavior in One Test ≠ Universal Software Behavior.

Expected–Observed Correspondence

TCMM shall evaluate the relationship between expected and observed software

behavior.

Possible determinations include:

• Corresponding

• Deviating

• Partially corresponding

• Inconclusive

• Invalidly assessed

• Not observed

Correspondence shall remain bounded by the actual test conditions.

Expected–Observed Correspondence ≠ Complete Software Correctness.

Test Result Classification

TCMM shall support identifiable result classifications.

Possible results include:

Passed: Observed evidence satisfies the applicable criterion within the assessed scope.

Failed: Valid evidence demonstrates non-satisfaction of the applicable

test condition.

Inconclusive: Available evidence does not support a sufficient determination.

Invalid: The execution or evaluation conditions prevent competent

interpretation.

Interrupted: The test did not reach its required completion condition.

Not Executed: The planned test was not performed.

Conditionally Passed: The observed result satisfies explicitly limited

conditions, where such classification is permitted by the governed criterion.

Result classification shall remain distinct from overall quality acceptance.

Passed Test ≠ Accepted Software Quality State.

Test Result Verification

TCMM shall assess whether reported test results correspond to actual

execution evidence and applicable evaluation rules.

Verification may include:

• Execution record correspondence

• Artifact identity confirmation

• Configuration confirmation

• Expected–observed comparison

• Evaluation rule application

• Evidence integrity assessment

A test framework's reported status shall not automatically establish a

verified result.

Reported Test Pass ≠ Verified Test Pass.

Test Repeatability

TCMM shall recognize the ability to repeat material test executions under

sufficiently comparable conditions.

Repeatability may depend on:

• Test identity

• Input preservation

• Environment conditions

• Artifact identity

• Configuration

• Dependencies

• Evaluation method

Repeatability shall remain distinguishable from identical outcomes.

Repeatable Test Execution ≠ Identical Test Outcome.

Test Determinism

TCMM shall identify whether relevant testing outcomes are expected to be deterministic.

Non-determinism may arise from:

• Concurrency

• Timing

• Randomization

• External services

• Distributed execution

• Environment variation

• Data ordering

A non-deterministic test shall not automatically be treated as invalid.

Its uncertainty and interpretation conditions must remain explicit.

Non-Deterministic Test ≠ Automatically Invalid Test.

Test Flakiness

TCMM shall recognize tests producing inconsistent outcomes under conditions

intended to be equivalent.

Flakiness may indicate:

• Timing sensitivity

• Environment instability

• Shared-state interference

• Test implementation defects

• Uncontrolled dependencies

• Actual intermittent software behavior

A flaky test shall not automatically establish that the software is correct or defective.

Flaky Test ≠ Proven Software Defect.

Test Coverage Basis

TCMM shall establish the basis on which material test coverage is measured or claimed.

Coverage may concern:

• Requirements

• Functions

• Statements

• Branches

• Paths

• Components

• Interfaces

• Scenarios

• Environments

• Failure conditions

Coverage claims shall preserve their measurement basis.

Coverage Percentage Without Coverage Definition ≠ Meaningful Coverage

Claim.

Requirements Coverage

TCMM shall assess which governed quality requirements are addressed by executed tests.

Requirements may be classified as:

• Covered by valid evidence

• Partially covered

• Associated with tests but not executed

• Not covered

• Unresolved

A test reference alone shall not establish requirements coverage.

Requirement-to-Test Link ≠ Executed Requirement Coverage.

Functional Coverage

TCMM shall assess the scope of software functionality evaluated by testing.

Functional coverage may concern:

• Required operations

• Input variations

• Output conditions

• Error handling

• Relevant workflows

• Boundary behavior

Functional coverage shall remain distinct from source-code coverage.

Code Coverage ≠ Functional Coverage.

Code Coverage

TCMM shall govern claims concerning the execution of source-code elements

during testing.

Code coverage may include:

• Statement coverage

• Line coverage

• Function coverage

• Branch coverage

• Condition coverage

Code coverage measurements shall preserve the relevant artifact,

instrumentation method, and exclusions.

Execution of code does not independently establish correct behavior.

Executed Code ≠ Verified Correct Code.

Branch and Condition Coverage

TCMM shall recognize coverage of relevant branching and conditional

behavior.

Coverage may concern:

• Decision outcomes

• Logical conditions

• Branch alternatives

• Relevant control-flow combinations

High branch coverage shall not establish that every meaningful input combination or failure condition has been assessed.

Complete Branch Coverage ≠ Complete Behavioral Coverage.

Integration Coverage

TCMM shall assess the extent to which relevant software integrations have

been evaluated.

Integrations may involve:

• Internal components

• APIs

• Databases

• External services

• Messaging systems

• Infrastructure dependencies

Simulated integrations shall remain distinguishable from actual integration

behavior.

Mocked Integration Test ≠ Verified Real Integration.

Scenario Coverage

TCMM shall assess the extent to which relevant operating scenarios have

been evaluated.

Scenarios may include:

• Normal use

• Peak workload

• Invalid inputs

• Recovery

• Concurrency

• Integration failure

• Relevant user workflows

Scenario coverage shall remain bounded by the defined scenario set.

All Defined Scenarios Tested ≠ All Relevant Scenarios Identified.

Environment Coverage

TCMM shall assess the environments and configurations represented by

material testing evidence.

Coverage may concern:

• Operating systems

• Hardware

• Runtime versions

• Configuration combinations

• Dependency versions

• Network conditions

• Deployment-relevant environments

Environment coverage shall not be generalized beyond assessed combinations.

Testing One Environment ≠ Testing Every Supported Environment.

Failure-Condition Coverage

TCMM shall assess whether relevant failure conditions have been included in

testing.

Conditions may include:

• Dependency failure

• Network interruption

• Resource exhaustion

• Invalid input

• Timeout

• Partial service unavailability

• Recovery failure

Failure-condition coverage shall remain connected to applicable quality

criteria.

Normal-Operation Test Success ≠ Verified Failure Resilience.

Coverage Measurement Governance

TCMM shall preserve the methods used to measure material coverage claims.

Measurements should identify:

• Coverage type

• Measurement method

• Covered population

• Excluded population

• Relevant artifact

• Execution evidence

• Aggregation rules

• Known limitations

Coverage measurements shall not be represented without sufficient

contextual meaning.

Measured Coverage ≠ Demonstrated Quality Sufficiency.

Coverage Interpretation

TCMM shall assess what a coverage result actually demonstrates.

A reported 95% line coverage may establish that defined instrumentation observed execution of 95% of included lines.

It does not establish:

• Correct assertions

• Representative test inputs

• Adequate failure testing

• Complete requirements coverage

• Correct integration behavior

• Production reliability

High Numerical Coverage ≠ High Evidentiary Quality.

Coverage Exclusions

TCMM shall identify material exclusions affecting coverage claims.

Exclusions may include:

• Generated code

• Unreachable code

• Unsupported platforms

• External integrations

• Specific modules

• Failure scenarios

• Environment combinations

Exclusions shall remain explicit where they affect interpretation.

Coverage After Exclusions ≠ Coverage of the Entire Software Object.

Coverage Gap

A Coverage Gap exists where relevant software behavior, requirements, or conditions remain insufficiently addressed by competent testing evidence.

Examples may include:

• Untested requirements

• Missing integration scenarios

• Unassessed failure conditions

• Unsupported environment combinations

• Missing regression coverage

• Incomplete workload testing

A coverage gap shall remain distinguishable from a demonstrated software

defect.

Coverage Gap ≠ Proven Software Failure.

Critical Coverage Gap

A Critical Coverage Gap exists where missing or insufficient testing prevents competent assessment of a materially consequential quality condition.

Examples may include:

• A critical financial calculation without suitable boundary testing

• A high-impact recovery mechanism without failure-condition testing

• A critical integration assessed only through unsuitable mocks

• A release candidate whose relevant artifact was not tested

• A safety-relevant function lacking competent behavioral evidence

TCMM shall preserve the critical gap for competent downstream assessment.

It does not independently determine software shutdown, release rejection, security compromise, or legal liability.

Critical Coverage Gap ≠ Automatic Software Failure.

Test Evidence Competence

TCMM shall assess whether evidence is competent to support the relevant testing claim.

Competence may depend on:

• Suitable test method

• Correct artifact

• Relevant configuration

• Appropriate environment

• Valid inputs

• Applicable evaluation rules

• Sufficient observations

• Relevant coverage

A test report may be authentic while insufficient for the quality claim being made.

Authentic Test Evidence ≠ Sufficient Test Evidence.

Test Evidence Integrity

TCMM shall preserve the integrity of material testing evidence.

Relevant considerations may include:

• Evidence source

• Execution identity

• Content integrity

• Artifact correspondence

• Configuration correspondence

• Temporal relevance

• Record preservation

• Material contradictions

An intact execution log shall not independently establish that the test was suitable or correctly interpreted.

Intact Test Log ≠ Verified Test Conclusion.

Testing Uncertainty

TCMM shall preserve material uncertainty concerning testing activities or

results.

Uncertainty may arise where:

• Execution evidence is incomplete

• Test conditions are unclear

• Artifact identity is unresolved

• Test oracle validity is uncertain

• Results are inconsistent

• Coverage limitations are material

• Assessment methods are insufficient

Uncertainty shall not be silently converted into a passing result.

Unresolved Test Evidence ≠ Verified Test Success.

Testing Conflict

A Testing Conflict exists where material testing records or conclusions are incompatible.

Examples may include:

• Contradictory execution results

• Conflicting artifact references

• Incompatible coverage reports

• Different outcomes under allegedly equivalent conditions

• Contradictory test environment records

• Conflicting result classifications

TCMM shall preserve the conflict and its supporting evidence.

Conflicting Test Results ≠ Permission to Select the Preferred Result.

Regression Testing Governance

TCMM shall govern testing intended to assess whether software changes

adversely affect previously established behavior.

Regression testing shall preserve:

• Relevant historical behavior

• Changed software artifact

• Applicable configuration

• Selected regression scope

• Execution evidence

• Observed results

• Coverage limitations

Passing selected regression tests shall not establish the absence of all

regressions.

Regression Test Success ≠ Absence of Regression Risk.

Testing Continuity

TCMM shall preserve material testing relationships across software

lifecycle changes.

Continuity may involve:

• Historical test cases

• Test revisions

• Artifact changes

• Configuration changes

• Dependency changes

• Updated criteria

• Coverage evolution

• Regression history

• Evidence preservation

Historical testing evidence shall not automatically establish current

software quality.

Previously Tested Software ≠ Currently Tested Software.

Testing Reassessment

TCMM shall support reassessment when material testing conditions change.

Triggers may include:

• New software versions

• Configuration changes

• Dependency updates

• Revised quality criteria

• New operating environments

• Newly discovered defects

• Contradictory test results

• Changed test methods

• New coverage requirements

Reassessment shall preserve relevant historical determinations.

Testing Reassessment ≠ Historical Testing Record Erasure.

Testing Governance States

TCMM shall support identifiable governance states.

Possible states include:

Identified: A testing governance object has been recognized.

Planned: Material testing objectives and scope have been established.

Designed: Relevant test cases and methods have been specified.

Readiness-Pending: Required execution preconditions remain unresolved.

Ready: Material execution preconditions are sufficiently established.

Execution-Pending: Testing has not yet been performed.

Executing: Testing is underway.

Completed: The defined execution completion conditions have been reached.

Passed: Valid evidence supports satisfaction of the applicable test

conditions.

Failed: Valid evidence demonstrates non-satisfaction of the applicable

test conditions.

Interrupted: Execution ended before required completion.

Invalid: Material execution or assessment conditions prevent competent

interpretation.

Inconclusive: Evidence is insufficient for a reliable determination.

Partially Covered: Relevant testing scope has been assessed only in part.

Coverage-Gap: Material coverage remains insufficient.

Critical-Coverage-Gap: Missing testing evidence prevents competent

assessment of a materially consequential condition.

Evidence-Limited: Supporting evidence is insufficient.

Conflict-Detected: Material testing records or conclusions are incompatible.

Superseded: A later testing assessment has replaced the earlier reference.

Reassessment-Required: Material changes require renewed testing

evaluation.

Unresolved: Available information does not support a sufficiently determined state.

Testing execution states, test result states, and coverage states shall

remain distinguishable even where represented together.

Test Passed ≠ Testing Governance Complete.

Governed Software Testing & Coverage State

The principal output of TCMM is:

Governed Software Testing & Coverage State

The state shall preserve sufficient information to identify:

Testing Governance Object: The governed testing activity or assessment.

Testing Identity: The identifiable testing reference.

Testing Purpose: The reason testing was performed.

Testing Scope: The applicable assessment boundaries.

Software Object: The software being evaluated.

Artifact Identity: The tested artifact.

Artifact Version: The relevant version.

Configuration Reference: The tested configuration.

Quality Requirements: The applicable governed requirements.

Quality Criteria: The criteria used for evaluation.

Test Strategy: The governing testing approach.

Test Classification: The relevant testing type.

Test Objectives: The intended assessment outcomes.

Test Designs: The defined testing methods and cases.

Test Case Identities: The identifiable test definitions.

Test Scenarios: The evaluated operating scenarios.

Test Inputs: The relevant execution inputs.

Test Data: The data used and its limitations.

Expected Behavior: The applicable expected software behavior.

Test Oracle: The mechanism used to evaluate observed behavior.

Test Preconditions: The required execution conditions.

Test Environment: The actual testing environment.

Environment Suitability: The suitability of that environment.

Environment Correspondence: The relationship to relevant operational

conditions.

Test Dependencies: Material supporting dependencies.

Execution Authority: The legitimate basis for testing actions.

Automated Testing Authority: Relevant automated action boundaries.

Execution Identities: The identifiable test runs.

Execution Conditions: The actual execution circumstances.

Execution Evidence: The records supporting execution claims.

Completion State: The recognized execution completion condition.

Interruption State: Relevant interrupted executions.

Failure State: Material failed tests.

Invalidity State: Invalid testing evidence or results.

Observed Behavior: The behavior actually recorded.

Behavior Correspondence: The relationship between expected and observed

behavior.

Test Result Classification: The recognized execution outcomes.

Test Result Verification: The competence of reported results.

Test Repeatability: Relevant repeated execution conditions.

Test Determinism: The recognized determinism conditions.

Test Flakiness: Material inconsistent testing outcomes.

Coverage Basis: The definition of coverage.

Requirements Coverage: The assessed requirement coverage.

Functional Coverage: The assessed functional behavior.

Code Coverage: The relevant source-code coverage.

Branch Coverage: The relevant decision coverage.

Integration Coverage: The assessed integration conditions.

Scenario Coverage: The assessed scenarios.

Environment Coverage: The assessed environments.

Failure-Condition Coverage: The assessed failure conditions.

Coverage Measurements: The recorded coverage values.

Coverage Interpretation: The justified meaning of those values.

Coverage Exclusions: Material excluded conditions.

Coverage Gaps: Missing relevant testing coverage.

Critical Coverage Gaps: Material gaps requiring competent attention.

Evidence Competence: The suitability of testing evidence.

Evidence Integrity: The condition of supporting evidence.

Testing Uncertainty: Material unresolved testing conditions.

Testing Conflicts: Contradictory evidence or conclusions.

Regression Testing: Relevant regression assessment evidence.

Testing Continuity: Preservation of lifecycle relationships.

Testing Reassessment: Material reassessment history.

Testing Governance State: The recognized testing condition.

Downstream References: Information required by DFM, QCAM, and parent SQS

governance.

The state shall not independently establish complete software correctness,

defect resolution, quality acceptance, software security, release authorization, or deployment authorization.

Minimum Implementation Framework

A conforming implementation of TCMM must provide the following six functional capabilities.

• Testing Object, Purpose, Scope, Artifact, Configuration, and Quality

Criteria Governance

Capability to establish identifiable testing governance objects, define

testing purpose and scope, associate testing activities with the actual software artifact and configuration, and preserve correspondence with governed quality requirements and criteria.

The implementation must distinguish the software intended for testing from the software actually tested.

• Test Strategy, Design, Scenario, Input, Data, Oracle, and Environment

Governance

Capability to establish suitable testing strategies, identifiable test

cases, scenarios, inputs, data conditions, expected behavior, test oracles, preconditions, environments, and supporting dependencies.

The implementation must distinguish test design from execution and test oracle availability from oracle validity.

• Test Authority, Execution, Observation, Result Classification, and

Verification Governance

Capability to preserve competent test execution authority, automated

testing boundaries, execution identities, actual execution conditions, execution evidence, observed behavior, expected–observed correspondence, result classifications, and result verification.

The implementation must distinguish reported test success from verified execution evidence and legitimate quality conclusions.

• Testing Coverage Definition, Measurement, Interpretation, Exclusion, and

Gap Governance

Capability to establish coverage bases, assess requirements, functional,

code, branch, integration, scenario, environment, and failure-condition coverage, interpret coverage measurements, identify exclusions, and recognize material and critical coverage gaps.

The implementation must distinguish numerical coverage from sufficient quality evidence.

• Testing Evidence, Repeatability, Flakiness, Conflict, Uncertainty,

Regression, and Continuity Governance

Capability to preserve competent and integrity-supported testing evidence,

identify repeatability and determinism conditions, recognize flaky tests, detect conflicts and uncertainty, assess regression testing, and maintain lifecycle continuity and reassessment.

The implementation must distinguish test evidence limitations from demonstrated software defects.

• Governed Software Testing & Coverage State and SQS Handoff Governance

Capability to produce a Governed Software Testing & Coverage State with

sufficient testing identity, artifact correspondence, configuration, quality criteria, execution evidence, observed behavior, result verification, coverage scope, exclusions, gaps, uncertainty, and continuity information for competent downstream DFM, QCAM, and parent SQS governance.

The implementation must not represent passing tests or high coverage as complete software quality validation. GIE™ Compatibility

TCMM is compatible with GIE™ as a downstream methodological analysis and execution layer.

GIE™ may support:

• Testing governance object identification

• Testing purpose and scope analysis

• Software object correspondence

• Artifact and version correspondence assessment

• Configuration correspondence analysis

• Quality requirement and criterion mapping

• Test strategy assessment

• Test classification

• Test objective analysis

• Test design evaluation

• Test case identity mapping

• Test scenario analysis

• Test input and data assessment

• Expected behavior identification

- Test oracle suitability analysis
- Test precondition assessment
- Test environment suitability analysis
- Environment correspondence evaluation
- Test dependency identification
- Test execution authority assessment
- Automated testing action boundary analysis
- Test execution identity mapping
- Execution condition assessment
- Execution evidence verification
- Test completion and interruption analysis
- Test failure interpretation
- Test invalidity detection
- Observed behavior analysis
- Expected–observed correspondence
- Test result classification
- Test result verification
- Test repeatability assessment
- Test determinism analysis
- Test flakiness detection
- Coverage basis identification
- Requirements coverage analysis
- Functional coverage analysis
- Code and branch coverage interpretation
- Integration coverage assessment
- Scenario coverage analysis
- Environment coverage assessment
- Failure-condition coverage analysis
- Coverage measurement interpretation
- Coverage exclusion identification
- Coverage gap detection
- Critical coverage gap detection
- Test evidence competence assessment
- Test evidence integrity assessment
- Testing uncertainty recognition
- Testing conflict detection

• Regression testing analysis

• Testing continuity mapping

• Testing reassessment support

• Governed testing and coverage state representation

GIE™ may identify untested quality conditions, detect inconsistencies between test reports and execution evidence, propose additional test scenarios, or identify possible coverage gaps.

Such outputs shall remain distinguishable from completed and verified

testing activities.

GIE™ must not fabricate test execution evidence, invent observed software behavior, represent generated test cases as executed tests, infer software correctness from numerical coverage, or treat autonomous testing authority as unrestricted authority over test criteria, software artifacts, or external systems.

OOF® logic adapts to the governed reality. The engine adapts to OOF®

logic.

Governance Logic

The canonical TCMM governance logic is:

Receive Governed Software Quality Requirements State

↓

Receive Governed Quality Criteria Definition State

↓

Identify Testing Governance Object

↓

Establish Testing Purpose and Scope

↓

Identify Software Object, Artifact, and Version

↓

Establish Configuration Correspondence

↓

Associate Testing with Quality Requirements and Criteria

↓

Define Test Strategy and Classification

↓

Establish Test Objectives

↓

Define Test Cases and Scenarios

↓

Identify Test Inputs and Data

↓

Establish Expected Behavior and Test Oracles

↓

Identify Test Preconditions

↓

Establish Test Environment and Suitability

↓

Assess Environment Correspondence

↓

Identify Test Dependencies

↓

Establish Test Execution Authority

↓

Establish Automated Testing Action Boundaries

↓

Identify Test Execution Instances

↓

Preserve Actual Execution Conditions

↓

Execute or Evaluate Material Testing Activities

↓

Preserve Test Execution Evidence

↓

Determine Test Completion, Interruption, or Invalidity

↓

Observe Software Behavior

↓

Compare Expected and Observed Behavior

↓

Classify Test Results

↓

Verify Reported Results

↓

Assess Repeatability, Determinism, and Flakiness

↓

Establish Coverage Basis

↓

Assess Requirements, Functional, Code, Branch, Integration, Scenario,

Environment, and Failure-Condition Coverage

↓

Interpret Coverage Measurements

↓

Identify Coverage Exclusions

↓

Identify Coverage Gaps and Critical Coverage Gaps

↓

Assess Testing Evidence Competence and Integrity

↓

Identify Testing Uncertainty and Conflicts

↓

Assess Relevant Regression Testing

↓

Determine Testing Governance State

↓

Establish Governed Software Testing & Coverage State

↓

Handoff to DFM, QCAM, and Parent SQS Governance

↓

Preserve Testing Continuity

↓

Reassess When Material Testing Conditions Change

The principal output is:

Governed Software Testing & Coverage State

Governance Outputs

TCMM may produce:

Testing Governance Object

Testing Identity Record

Testing Purpose Record

Testing Scope Record

Software Object Correspondence Record

Tested Artifact Identity Record

Tested Artifact Version Record

Tested Configuration Correspondence Record

Quality Requirement Testing Correspondence Record

Quality Criterion Testing Correspondence Record

Test Strategy Record

Test Classification Record

Test Objective Record

Test Design Record

Test Case Register

Test Scenario Register

Test Input Record

Test Data Record

Expected Behavior Record

Test Oracle Record

Test Preconditions Record

Test Environment Record

Test Environment Suitability Assessment

Environment Correspondence Assessment

Test Dependency Record

Test Execution Authority Record

Automated Testing Authority Record

Test Execution Identity Record

Test Execution Conditions Record

Test Execution Evidence Register

Test Completion Record

Test Interruption Record

Test Failure Record

Test Invalidity Record

Observed Software Behavior Record

Expected–Observed Correspondence Assessment

Test Result Classification Record

Test Result Verification Record

Test Repeatability Assessment

Test Determinism Assessment

Test Flakiness Record

Test Coverage Basis Record

Requirements Coverage Record

Functional Coverage Record

Code Coverage Record

Branch Coverage Record

Integration Coverage Record

Scenario Coverage Record

Environment Coverage Record

Failure-Condition Coverage Record

Coverage Measurement Record

Coverage Interpretation Record

Coverage Exclusion Record

Coverage Gap

Critical Coverage Gap

Test Evidence Competence Assessment

Test Evidence Integrity Assessment

Testing Uncertainty State

Testing Conflict

Regression Testing Record

Testing Continuity Record

Testing Reassessment Record

Testing Governance State

SQS Testing & Coverage Handoff Record

The principal output is:

Governed Software Testing & Coverage State

Critical Distinctions

TCMM preserves the following distinctions:

Test Execution Success ≠ Demonstrated Software Quality

Testing Governance Object ≠ Test Execution Report

Testing Purpose ≠ Testing Result

Defined Testing Scope ≠ Executed Testing Scope

Testing One Software Component ≠ Testing the Entire Software System

Declared Tested Version ≠ Verified Tested Artifact

Testing Version A ≠ Verification of Version B

Tested Configuration ≠ Every Configuration

Requirement Linked to Test ≠ Requirement Verified by Test

Executing a Test Against Modified Criteria ≠ Satisfying the Original Criteria

Test Strategy Approval ≠ Test Strategy Execution

Test Classification ≠ Test Competence

Test Objective ≠ Test Outcome

Designed Test ≠ Executed Test

Test Case Identity ≠ Test Execution Identity

Defined Test Scenario ≠ Observed Scenario Behavior

Defined Test Input ≠ Verified Executed Input

Available Test Data ≠ Representative Test Data

Expected Test Result ≠ Observed Test Result

Test Oracle Availability ≠ Test Oracle Validity

Declared Test Preconditions ≠ Verified Test Readiness

Documented Test Environment ≠ Actual Test Environment

Operational Test Environment ≠ Suitable Assessment Environment

Test Environment Similarity ≠ Demonstrated Operational Equivalence

Available Test Dependency ≠ Suitable Test Dependency

Test Execution Authority ≠ Quality Acceptance Authority

Testing Task Authority ≠ Constituent Testing Action Authority

Same Test Case ≠ Same Test Execution

Planned Test Conditions ≠ Actual Test Conditions

Test Report Generation ≠ Verified Test Execution

Completed Test ≠ Passed Test

Interrupted Test ≠ Failed Software Quality Criterion

Failed Test ≠ Verified Software Defect

Invalid Test Result ≠ Valid Test Failure

Observed Behavior in One Test ≠ Universal Software Behavior

Expected–Observed Correspondence ≠ Complete Software Correctness

Passed Test ≠ Accepted Software Quality State

Reported Test Pass ≠ Verified Test Pass

Repeatable Test Execution ≠ Identical Test Outcome

Non-Deterministic Test ≠ Automatically Invalid Test

Flaky Test ≠ Proven Software Defect

Coverage Percentage Without Coverage Definition ≠ Meaningful Coverage

Claim

Requirement-to-Test Link ≠ Executed Requirement Coverage

Code Coverage ≠ Functional Coverage

Executed Code ≠ Verified Correct Code

Complete Branch Coverage ≠ Complete Behavioral Coverage

Mocked Integration Test ≠ Verified Real Integration

All Defined Scenarios Tested ≠ All Relevant Scenarios Identified

Testing One Environment ≠ Testing Every Supported Environment

Normal-Operation Test Success ≠ Verified Failure Resilience

Measured Coverage ≠ Demonstrated Quality Sufficiency

High Numerical Coverage ≠ High Evidentiary Quality

Coverage After Exclusions ≠ Coverage of the Entire Software Object

Coverage Gap ≠ Proven Software Failure

Critical Coverage Gap ≠ Automatic Software Failure

Authentic Test Evidence ≠ Sufficient Test Evidence

Intact Test Log ≠ Verified Test Conclusion

Unresolved Test Evidence ≠ Verified Test Success

Conflicting Test Results ≠ Permission to Select the Preferred Result

Regression Test Success ≠ Absence of Regression Risk

Previously Tested Software ≠ Currently Tested Software

Testing Reassessment ≠ Historical Testing Record Erasure

Test Passed ≠ Testing Governance Complete

Cross-Architecture Boundary

TCMM governs software-quality-specific testing identity, planning, design,

execution, observation, result verification, coverage, evidence, limitations, and continuity within SQS and SLGA™.

It does not independently establish software quality requirements, redefine quality criteria, determine complete software quality, resolve defects, authorize quality acceptance, certify security, authorize software release, or authorize deployment.

Within SQS:

QRM — Quality Requirements Module governs software quality requirement

establishment.

QCDM — Quality Criteria Definition Module governs quality criterion

definition, applicability, assessment methods, and evidence requirements.

TCMM — Testing & Coverage Management Module governs actual testing

activities, execution evidence, observed results, and coverage.

DFM — Defect Management Module governs software defect identification,

classification, impact, correction references, and resolution verification.

QCAM — Quality Conformity & Acceptance Module governs quality conformity

assessment, validation, acceptance authority, acceptance decisions, and residual limitations.

Within SLGA™:

Software Requirements Standard (SRS) governs the broader software

requirements lifecycle.

Software Creation Standard (SCS) governs software creation.

Software Dependency Standard (SDS) governs software dependencies.

Software Configuration Standard (SCFS) governs software configuration.

Software Quality Standard (SQS) governs software quality.

Software Security Standard (SSS) governs software security.

Software Release Standard (SRLS) governs software release.

Software Deployment Standard (SDPS) governs software deployment.

Software Change Management Standard (SCMS) governs broader lifecycle

changes.

Software Retirement Standard (SRTS) governs software retirement.

Software Lifecycle Control Standard (SLCS) governs cross-cutting

lifecycle control.

Relevant interoperating OOF® architectures include:

GOA™ — Governance Architecture: General authority and delegation

governance.

AGA™ — Accountability Governance Architecture: Relevant responsibility

and accountability relationships.

ORA™ — Operational Reality Architecture: Competent assessment of actual

software behavior and operating conditions.

ASGA™ — Autonomous Systems Governance Architecture: Relevant autonomous

testing activities and execution boundaries.

AGI™ — Agent Governance Infrastructure: Relevant agent permissions,

tool-path authority, and constituent action governance.

TCMM preserves testing-and-coverage-specific governance without replacing

these competent domains.

Canonical Boundary

TCMM begins when a governed software quality criterion requires testing, execution evidence, observed behavior, coverage assessment, or testing reassessment.

TCMM governs:

• Testing governance objects

• Testing identity

• Testing purpose

• Testing scope

• Software object correspondence

• Artifact identity

• Artifact version correspondence

• Configuration correspondence

• Quality requirement correspondence

• Quality criterion correspondence

• Test strategies

• Test classifications

• Test objectives

• Test designs

• Test cases

• Test scenarios

• Test inputs

• Test data

• Expected behavior

• Test oracles

• Test preconditions

• Test environments

• Environment suitability

• Environment correspondence

- Test dependencies
- Test execution authority
- Automated testing authority
- Test execution identities
- Execution conditions
- Execution evidence
- Test completion
- Test interruption
- Test failure
- Test invalidity
- Observed software behavior
- Expected–observed correspondence
- Test result classifications
- Test result verification
- Test repeatability
- Test determinism
- Test flakiness
- Coverage bases
- Requirements coverage
- Functional coverage
- Code coverage
- Branch coverage
- Integration coverage
- Scenario coverage
- Environment coverage
- Failure-condition coverage
- Coverage measurements
- Coverage interpretation
- Coverage exclusions
- Coverage gaps
- Critical coverage gaps
- Test evidence competence
- Test evidence integrity
- Testing uncertainty
- Testing conflicts
- Regression testing

• Testing continuity

• Testing reassessment

• Testing governance states

TCMM does not independently determine:

• Complete software requirements validity

• Software creation authorization

• Dependency suitability

• Configuration baseline establishment

• Quality criterion establishment

• Complete software correctness

• Defect root-cause attribution

• Defect resolution

• Software quality acceptance

• Software security

• Release authorization

• Deployment authorization

• Legal compliance

• Legal liability

TCMM ends when material testing and coverage conditions have been represented as a:

Governed Software Testing & Coverage State

with sufficient testing identity, artifact correspondence, configuration,

quality criteria, execution evidence, observed behavior, result verification, coverage scope, exclusions, gaps, uncertainty, and continuity information for competent downstream DFM, QCAM, and parent SQS governance.

Use Case 1 — An AI-Generated Test Suite Reports 98% Coverage but Misses

a Critical Failure Scenario

An organization uses an AI coding assistant to develop a financial

calculation service.

The assistant generates:

• Application code

• Unit tests

• Test data

• Expected outputs

• Coverage configuration

The automated pipeline reports:

• All tests passed

• 98% line coverage

• No detected defects

The development team interprets the result as evidence that the service is production-ready.

However, the generated tests primarily evaluate normal input values.

The test suite does not adequately assess:

• Extreme numerical values

• Rounding boundaries

• Missing transaction data

• Repeated transactions

• Relevant concurrency conditions

• Critical failure recovery

The expected outputs were also generated by the same AI system that produced the implementation, without sufficient independent verification.

TCMM evaluates:

Governed Quality Criteria → Test Design → Test Oracle → Executed Inputs →

Observed Behavior → Coverage Basis → Coverage Interpretation → Coverage Gaps

The assessment identifies:

• Valid execution evidence for the tests actually performed

• High line coverage under the selected instrumentation

• Insufficient evidence of independent test oracle validity

• Incomplete boundary-condition coverage

• Missing critical failure scenarios

• A material difference between code coverage and meaningful behavioral

coverage.

The resulting testing state may recognize:

Passed

for the valid executed tests, while separately recording:

Critical-Coverage-Gap

for the unassessed consequential conditions.

The critical distinctions are:

Executed Code ≠ Verified Correct Code.

High Numerical Coverage ≠ High Evidentiary Quality.

Test Oracle Availability ≠ Test Oracle Validity.

All Defined Scenarios Tested ≠ All Relevant Scenarios Identified.

The result: TCMM prevents high numerical test coverage and successful

automated execution from being interpreted as comprehensive software quality evidence when material behavior remains unassessed.

Use Case 2 — A Testing Agent Reports Success After Testing the Wrong

Software Artifact

An organization prepares a new software release.

The release candidate contains:

• Updated application code

• A revised dependency

• Modified runtime configuration

• A new build artifact

An autonomous testing agent is instructed to execute the regression suite against the release candidate.

The agent selects an available test environment and executes the tests.

The pipeline reports:

• All regression tests passed

• No critical test failures

• Successful completion

However, the environment still contains the previous software artifact.

The agent executed the correct test suite but against the wrong version.

The test report contains the expected release version label, but the actual

artifact identity was not verified.

TCMM evaluates:

Testing Objective → Artifact Identity → Configuration → Execution

Environment → Test Execution → Evidence → Reported Result

The assessment identifies:

• A legitimate testing objective

• Successful execution of the selected regression tests

• Insufficient correspondence between the declared and actual tested

• Testing evidence applicable to the previous software version

• No competent testing evidence establishing the release candidate's

artifact. regression state.

The relevant testing governance state may be:

Invalid

for the intended release-candidate assessment, or:

Evidence-Limited

where artifact correspondence cannot be sufficiently established.

The critical distinctions are:

Declared Tested Version ≠ Verified Tested Artifact.

Testing Version A ≠ Verification of Version B.

Reported Test Pass ≠ Verified Test Pass.

Test Execution Success ≠ Demonstrated Software Quality.

The result: TCMM prevents an automated testing system from transferring

successful test results from one software artifact to another merely because the test report contains the intended version label.

Canonical Closing Statement

Software testing is not governed merely because a test suite exists, an automated pipeline completes, a dashboard reports successful execution, or a coverage percentage appears favorable.

A test may execute correctly against the wrong artifact.

A test may pass because its expected result is incorrect.

A test suite may achieve high code coverage while failing to evaluate material operational scenarios.

An integration test may succeed against a simulated service while the actual integration remains unverified.

And an autonomous testing agent may execute thousands of tests without establishing whether those tests provide competent evidence for the quality claims being made.

Test Execution Success ≠ Demonstrated Software Quality.

Designed Test ≠ Executed Test.

Reported Test Pass ≠ Verified Test Pass.

Executed Code ≠ Verified Correct Code.

High Numerical Coverage ≠ High Evidentiary Quality.

Testing Version A ≠ Verification of Version B.

These distinctions become increasingly important as software testing is generated, orchestrated, and interpreted by automated systems.

The number of executed tests may increase dramatically while the quality of the underlying evidence remains limited.

TCMM therefore governs not merely the existence of testing activity, but

the relationship between testing purpose, governed criteria, actual software artifacts, execution conditions, observed behavior, result interpretation, and the coverage those observations genuinely support.

It does not require every possible software behavior to be tested.

It requires material testing claims to remain bounded by the behavior

actually assessed.

It does not treat every failed test as proof of a software defect.

It requires failures, invalid executions, inconclusive results, and

environmental limitations to remain distinguishable.

It does not treat high coverage as sufficient evidence of software quality.

It requires coverage to remain connected to its definition, measurement

basis, exclusions, and material untested conditions.

TCMM transforms software testing from a collection of automated

executions, reports, and coverage metrics into a governed evidentiary process — ensuring that software organizations can establish not merely how many tests were executed or how much code was covered, but which software was actually evaluated, under which conditions, what behavior was observed, what the evidence demonstrates, and which quality conditions remain unverified.

Software testing is not governed because tests run successfully. It is governed when test identity, artifact correspondence, criteria, execution authority, observed behavior, evidence competence, coverage limitations, and lifecycle continuity remain sufficiently established to support competent software quality assessment.